

claude-windows / d4cf3fd6-b5bd-4ed1-9b57-88eea69333cd

Metadata

```
- Source: `claude-windows`
- Kind: `claude`
- Source file: `C:\Users\avidu\.claude\projects\C--Users-avidu-Projects-badminton-highlight-indexer\d4cf3fd6-b5bd-4ed1-9b57-88eea69333cd\subagents\workflows\wf_8c732492-138\agent-a6adca732bba00df3.jsonl`
- SHA-256: `8507ab5ed399fcf3ce7c71512865445e38ce753f362cab06b100be291b0dd0b3`
- Source modified: `2026-07-04T09:02:34+00:00`
- Imported at: `2026-07-05T16:48:23+00:00`
- project: `wf_8c732492-138`
- session_id: `d4cf3fd6-b5bd-4ed1-9b57-88eea69333cd`
```

Transcript

1. user (2026-07-04T08:55:05.787Z)

You are an ADVERSARIAL verifier for a fix to PR #468 (branch fix/drain-concurrency) of Khelsutra/badminton-highlight-indexer. A fix agent just left UNCOMMITTED changes in the worktree C:/Users/avidu/Projects/khelsutra-guru/wt-drain-concurrency. Your default stance: try to REFUTE that the fix is correct and complete.

The blocker it must resolve:

REVIEWER BLOCKER (verbatim, from PR #468):

"claim_due_job() is CAS-safe for avoiding double-claims, but under parallel drain workers the losers return None (backend/infrastructure/database_jobs.py:357-380). _drain_due_jobs() then treats None as 'queue empty' and exits (backend/api/job_worker.py:273-276). With max_workers > 1, multiple workers can select the same first queued row; one updates it, and the losers stop even though later jobs are still queued.

I reproduced locally on this head with 8 queued jobs and 8 concurrent claimers:

[None, None, None, None, 'j0', None, None, 'j1'] -> only 2/8 claims

For long Cloud Run bucket-poll jobs this can leave dispatch capacity unused and queued jobs waiting until another submit/wake or a running worker finally frees up, which undermines the concurrency improvement this PR is trying to provide.

Suggested fix: make claim_due_job() retry on rowcount != 1 until it either successfully claims a row or a fresh SELECT finds no runnable rows, or return a distinct lost-race sentinel so _drain_due_jobs() loops instead of treating it as empty. Please add a regression test with concurrent claimers/drains over N queued jobs proving there is no false-empty result / that N distinct jobs can be claimed."

The fix agent reported:

```
- files changed: ["C:/Users/avidu/Projects/khelsutra-guru/wt-drain-concurrency/backend/infrastructure/database_jobs.py", "C:/Users/avidu/Projects/khelsutra-guru/wt-drain-concurrency/tests/test_job_worker_loop.py"]
- tests added: ["tests/test_job_worker_loop.py :: test_concurrent_claimers_claim_all_jobs_distinctly"]
- suite: 1634 passed, 10 skipped, 8 warnings in 112.63s (0:01:52) | ruff: ruff 0.15.16: All checks passed! | mypy: mypy 2.1.0: Success: no issues found in 147 source files (4 pre-existing annotation-unchecked notes, unrelated to this change)
- summary: Implemented the reviewer's preferred fix: claim_due_job() in backend/infrastructure/database_jobs.py now loops on a lost CAS race ? re-SELECT the head runnable row; if none, return None (None now truly means nothing-runnable); attempt the CAS UPDATE; on rowcount==1 return the claimed job, otherwise retry. The loop is deliberately unbounded (every lost race means another worker claimed that row, so the runnable set shrinks ? lock-free global progress; a cap would reintroduce the false-empty), and it reuses the single _connect() connection safely because the SELECT is an autocommit read and each CAS commits before the next iteration, so no write lock is held across retries (the 30s busy_timeout covers momentary contention). The docstring explains the retry, the new None semantics, and cites the #466/#468 parallel-drain design. _drain_due_jobs() in backend/api/job_worker.py needed NO change and was not touched ? its "if job is None: break" is now correct because None means genuinely empty. Added the reviewer-required regression test: 8 queued jobs, 8 claimer threads released
```

together by a `threading.Barrier`, each calling `db.claim_due_job()` once; asserts all 8 results are non-None (no false-empty), the 8 claimed ids are exactly the 8 distinct queued jobs (no double-claim), all rows end up 'running', and a subsequent claim returns None (truly empty). Verified the test catches the pre-fix bug (5/5 failures with the fix stashed) and is stable with the fix (15/15 passes under concurrent sibling suites); no 'database is locked' errors surfaced, so no extra contention handling was needed beyond the existing `busy_timeout`.

- problems:

DO (read-only on the worktree except where stated):

1. Read the full diff: `git -C C:/Users/avidu/Projects/khelsutra-guru/wt-drain-concurrency diff` (plus `git -C C:/Users/avidu/Projects/khelsutra-guru/wt-drain-concurrency status --short` for new files). Check the fix logic line by line against the blocker. Hunt for: edge cases the reviewer's repro implies but the fix misses, behavior changes beyond the blocker's scope, style that clashes with the repo, docstrings that overstate.
2. PROVE the regression test bites: create a THROWAWAY detached worktree at the PR head: `git -C C:/Users/avidu/Projects/khelsutra-guru/wt-drain-concurrency worktree add C:/Users/avidu/AppData/Local/Temp/claude/C--Users-avidu-Projects-badminton-highlight-indexer/d4cf3fd6-b5bd-4ed1-9b57-88eea69333cd/scratchpad/verify-468 8534077b79794ecef7c4f6559a708c5bfe382498 --detach` ; copy ONLY the new/changed test file(s) from `C:/Users/avidu/Projects/khelsutra-guru/wt-drain-concurrency` into it; run those specific tests there with `python -m pytest <testfile> -q -p no:cacheprovider` ? they MUST FAIL (that proves the test catches the original bug). Then remove the throwaway: `git -C C:/Users/avidu/Projects/khelsutra-guru/wt-drain-concurrency worktree remove --force C:/Users/avidu/AppData/Local/Temp/claude/C--Users-avidu-Projects-badminton-highlight-indexer/d4cf3fd6-b5bd-4ed1-9b57-88eea69333cd/scratchpad/verify-468`. If the test PASSES at the unfixed head, that is a refutation ? report it.
3. Run the new tests in the REAL worktree to confirm they pass with the fix: `python -m pytest <testfile> -q -p no:cacheprovider`.
4. Do NOT modify the fix agent's changes in `C:/Users/avidu/Projects/khelsutra-guru/wt-drain-concurrency`. Do NOT commit or push anywhere. PowerShell tool on Windows; global python 3.13.
5. For PR #468 specifically (if that is your PR): also re-run the new concurrency test 5 times in a row in the real worktree to smoke out flakiness, and report.

Fill the schema honestly. `verdict='approve'` ONLY if the fix fully resolves the blocker, the test provably bites, and you found no real issues.

2. user (2026-07-04T08:55:14.397Z)

```
M backend/infrastructure/database_jobs.py
M tests/test_job_worker_loop.py
8534077 review: gate concurrency on effective compute path, not the server default (#466)
d56237f fix(cloud-serving): parallelize the drain under cloud_run (#466)
a5d6246 serving: re-queue interrupted compile jobs on restart (#329 reliability) (#460)
```

3. user (2026-07-04T08:55:20.313Z)

```
diff --git a/backend/infrastructure/database_jobs.py b/backend/infrastructure/database_jobs.py
index a9e18fb..8310f07 100644
--- a/backend/infrastructure/database_jobs.py
+++ b/backend/infrastructure/database_jobs.py
@@ -358,25 +358,38 @@ class DatabaseJobsMixin:
     """Atomically claim ONE runnable job ? 'queued', or 'waiting' whose ``next_poll_at`` is
    due ?
        flipping it to 'running'. Compare-and-set on (id, status) so concurrent workers never
        double-claim (the DB row-claim is the only coordination needed; no master). Earliest-
    due
    - first. Returns the claimed job dict, or None if nothing is runnable."""
    + first. Returns the claimed job dict, or None ONLY when a fresh SELECT finds nothing
    + runnable ? never for a lost race.
    +
    + A lost CAS RETRIES instead of returning: under the parallel drain (#466) several
    workers
    + can SELECT the same head-of-queue row, and if the losers reported None the caller
    + (``_drain_due_jobs``) would read it as "queue empty" and stop with runnable jobs still
    + queued ? stranding dispatch capacity until some later submit/wake. The retry loop is
```

```

+         deliberately UNBOUNDED: every lost race means another worker successfully claimed that
+         row, so the runnable set shrank and global progress is guaranteed (lock-free progress);
+         a retry cap would just reintroduce the false-empty. One connection serves all attempts
+         ?
+         the SELECT is an autocommit read and each CAS commits before the next iteration, so no
+         write lock is ever held across a retry (busy_timeout covers momentary contention)."""
+         try:
+             with self._connect() as conn:
+                 cur = conn.cursor()
+                 row = cur.execute(
+                     "SELECT id, status FROM jobs WHERE status='queued' "
+                     "OR (status='waiting' AND (next_poll_at IS NULL OR next_poll_at <=
CURRENT_TIMESTAMP)) "
+                     "ORDER BY COALESCE(next_poll_at, created_at) ASC LIMIT 1"
+                 ).fetchone()
+                 if not row:
+                     return None
+                 cur.execute(
+                     "UPDATE jobs SET status='running', started_at=CURRENT_TIMESTAMP "
+                     "WHERE id=? AND status=?",
+                     (row["id"], row["status"]),
+                 )
+                 conn.commit()
+                 if cur.rowcount != 1:
+                     return None # lost the race to a concurrent worker
+                 while True:
+                     row = cur.execute(
+                         "SELECT id, status FROM jobs WHERE status='queued' "
+                         "OR (status='waiting' AND (next_poll_at IS NULL OR next_poll_at <=
CURRENT_TIMESTAMP)) "
+                         "ORDER BY COALESCE(next_poll_at, created_at) ASC LIMIT 1"
+                     ).fetchone()
+                     if not row:
+                         return None # nothing runnable ? a TRUE empty, not a lost race
+                     cur.execute(
+                         "UPDATE jobs SET status='running', started_at=CURRENT_TIMESTAMP "
+                         "WHERE id=? AND status=?",
+                         (row["id"], row["status"]),
+                     )
+                     conn.commit()
+                     if cur.rowcount == 1:
+                         break # claimed it
+                     # Lost the race: the winner flipped this row; re-SELECT for the next one.
+                 return self.get_job(row["id"])
+             except Exception as e:
+                 logger.error(f"Failed to claim a due job: {e}")
diff --git a/tests/test_job_worker_loop.py b/tests/test_job_worker_loop.py
index e84d2a6..fe9c6f9 100644
--- a/tests/test_job_worker_loop.py
+++ b/tests/test_job_worker_loop.py
@@ -13,16 @@ Covered:
- a `JobWaiting` parks the job (no terminal state) and a later drain (once due) resumes it;
- not-yet-due parked jobs are left alone and a wake-timer is armed for them;
- the wake-scheduling decision (`_schedule_next_drain_if_waiting`) is asserted with the
- timer seam stubbed ? no real waiting, no leaked threads.
+ timer seam stubbed ? no real waiting, no leaked threads;
+- concurrent claimers (#466/#468 parallel drain) never double-claim AND never report a
+ false-empty: a lost CAS race retries inside `claim_due_job`, so None means truly empty.

```

The real-timer seam (`\_arm\_wake\_timer`) is monkeypatched to a recorder in every test that can park a job, so nothing actually sleeps and no daemon timer outlives the test.

```

import sqlite3
+import threading

```



```

334
335     def seconds_until_next_due(self) -> Optional[float]:
336         """Seconds until the soonest parked ('waiting') job becomes due, so the worker
337         can
338         schedule a wake-up (EXECUTION_POLICY.md P3 self-scheduling). Returns 0.0 if one
339         is
340         already due, or None if nothing is waiting. A NULL ``next_poll_at`` ? due now
341         (0.0)."""
342         try:
343             with self._connect() as conn:
344                 row = (
345                     conn.cursor()
346                     .execute(
347                         "SELECT MIN(CASE WHEN next_poll_at IS NULL THEN 0 ELSE "
348                         " (julianday(next_poll_at) - julianday('now')) * 86400.0 END)
349                         "FROM jobs WHERE status='waiting'"
350                     )
351                     .fetchone()
352                 )
353                 if row is None or row["secs"] is None:
354                     return None
355                 return max(0.0, float(row["secs"]))
356         except Exception as e:
357             logger.error(f"Failed to compute next-due delay: {e}")
358             return None
359
360     def claim_due_job(self) -> Optional[Dict[str, Any]]:
361         """Atomically claim ONE runnable job ? 'queued', or 'waiting' whose
362         ``next_poll_at`` is due ?
363         flipping it to 'running'. Compare-and-set on (id, status) so concurrent workers
364         never
365         double-claim (the DB row-claim is the only coordination needed; no master).
366         Earliest-due
367         first. Returns the claimed job dict, or None ONLY when a fresh SELECT finds
368         nothing
369         runnable ? never for a lost race.
370         A lost CAS RETRIES instead of returning: under the parallel drain (#466) several
371         workers
372         can SELECT the same head-of-queue row, and if the losers reported None the
373         caller
374         (``_drain_due_jobs``) would read it as "queue empty" and stop with runnable jobs
375         still
376         queued ? stranding dispatch capacity until some later submit/wake. The retry
377         loop is
378         deliberately UNBOUNDED: every lost race means another worker successfully
379         claimed that
380         row, so the runnable set shrank and global progress is guaranteed (lock-free
381         progress);
382         a retry cap would just reintroduce the false-empty. One connection serves all
383         attempts ?
384         the SELECT is an autocommit read and each CAS commits before the next iteration,
385         so no
386         write lock is ever held across a retry (busy_timeout covers momentary
387         contention)."""
388         try:
389             with self._connect() as conn:
390                 cur = conn.cursor()
391                 while True:
392                     row = cur.execute(
393                         "SELECT id, status FROM jobs WHERE status='queued' "
394                         "OR (status='waiting' AND (next_poll_at IS NULL OR next_poll_at
395                         <= CURRENT_TIMESTAMP)) "
396                         "ORDER BY COALESCE(next_poll_at, created_at) ASC LIMIT 1"

```

```

381         ).fetchone()
382     if not row:
383         return None # nothing runnable ? a TRUE empty, not a lost race
384     cur.execute(
385         "UPDATE jobs SET status='running', started_at=CURRENT_TIMESTAMP
386         "
387         "WHERE id=? AND status=?",
388         (row["id"], row["status"]),
389     )
390     conn.commit()
391     if cur.rowcount == 1:
392         break # claimed it
393     # Lost the race: the winner flipped this row; re-SELECT for the next
394     one.
395     return self.get_job(row["id"])
396 except Exception as e:
397     logger.error(f"Failed to claim a due job: {e}")
398     return None
399
400 def fail_stale_jobs(self, reason: str = "interrupted by server restart") -> int:
401     """Crash recovery: any job still queued/running from a previous process is dead
402     (the executor is in-process). Mark failed so clients see a terminal state, not a
403     hang. Returns the number of jobs swept.
404
405     NOTE: ``recover_stale_jobs`` is the preferred startup call ? it RE-QUEUES stale
406     ``compile`` jobs (idempotent CPU stitch, safe to retry) instead of failing them.
407     This
408     blanket-fail method stays for callers/tests that want the old behaviour."""
409     try:
410         with self._connect() as conn:
411             cur = conn.cursor()
412             cur.execute(

```

5. user (2026-07-04T08:55:35.634Z)

```

240     ) -> None:
241         """M11 data-flywheel (COMPUTE_DECOUPLED_SERVING M11, D12). DEFAULT-OFF + INERT: a
242         no-op unless
243         ``config.flywheel.enabled`` AND attested consent ? and consent is a **faked hard-
244         deny** today
245         (``flywheel.consent_attested`` returns False for every job until counsel + the
246         consent schema
247         land), so this captures **nothing** in production. When activated it captures the
248         consented job's
249         artifacts into the per-video workspace (? later labeling ? the golden training set).
250         Best-effort ? never fails a completed job because the flywheel hiccuped. ``job`` is
251         the claimed
252         row (carries ``owner_id``); ``result`` is the pipeline output."""
253         flywheel_cfg = getattr(config, "flywheel", None)
254         if not flywheel_cfg or not getattr(flywheel_cfg, "enabled", False):
255             return
256         try:
257             from backend import flywheel
258
259             flywheel.run_for_job(job, result, config)
260         except Exception as e: # never wedge a completed job on the flywheel
261             logger.warning("Flywheel for job %s failed (non-fatal): %s", job.get("id"), e)
262
263 def _drain_due_jobs(config: Any, db: Any) -> int:
264     """One worker tick: claim and run every currently-runnable job (queued, or waiting
265     whose
266     `next_poll_at` is due), earliest-due first, until the table has nothing runnable.

```

```

264
265     Each parked job (`JobWaiting` ? status 'waiting') drops out of this drain and is
picked
266     up by a FUTURE drain once due; we schedule that follow-up drain via
`_schedule_drain` so
267     a single-worker box keeps making progress with no central timer (the DB is the
clock).
268     Returns the number of jobs that reached a terminal/parked state this tick.
269     """
270     import backend.main as _main
271
272     ran = 0
273     while True:
274         job = db.claim_due_job()
275         if job is None:
276             break
277         ran += 1
278         _main._run_claimed_job(job, config, db)
279     # If anything is parked-but-not-yet-due, arrange a wake-up so it resumes without a
new
280     # submit. On the single-box executor this is a delayed re-drain; multi-worker just
relies
281     # on the next submit/drain. Best-effort ? never raises into the worker.
282     _main._schedule_next_drain_if_waiting(config, db)
283     return ran
284
285
286 def _schedule_next_drain_if_waiting(config: Any, db: Any) -> None:
287     """If any job is parked in 'waiting', arm a re-drain for when the soonest one is
due, so a
288     self-scheduled job resumes with no further client submit. The drain itself re-checks
289     due-ness, so an early wake is harmless and a clamped wake for a far-future job is
fine."""
290     import backend.main as _main
291
292     try:
293         delay = db.seconds_until_next_due()
294     except Exception as e: # pragma: no cover - defensive; never wedge the worker
295         logger.error(f"Could not compute next-due delay: {e}")
296         return
297     if delay is None:
298         return # nothing waiting
299     # Clamp so a far-future job doesn't pin a thread forever; the drain re-checks due-
ness.

```

6. user (2026-07-04T08:55:53.106Z)

```

1     """P3 worker self-scheduling loop (EXECUTION_POLICY.md P3).
2
3     The P2 slice added the DB primitives (`claim_due_job`, `set_job_waiting`,
4     `seconds_until_next_due`); this slice wires the `main.py` worker to them so the worker
5     DRAINS the jobs table via `claim_due_job` and honours WAIT_AND_RESUME: a job that raises
6     `JobWaiting` is parked (status 'waiting' + `next_poll_at` + progress) and re-claimed
when
7     due ? instead of the old one-shot `submit(_run_job, ...)`
8
9     Covered:
10    - `_drain_due_jobs` claims + runs queued jobs to a terminal state (no regression on the
11      existing sync/submit path ? same `done`/`failed`/`verdict` semantics);
12    - earliest-due ordering across queued + due-waiting;
13    - a `JobWaiting` parks the job (no terminal state) and a later drain (once due) resumes
it;
14    - not-yet-due parked jobs are left alone and a wake-timer is armed for them;
15    - the wake-scheduling decision (`_schedule_next_drain_if_waiting`) is asserted with the
16      timer seam stubbed ? no real waiting, no leaked threads;

```

```

17     - concurrent claimers (#466/#468 parallel drain) never double-claim AND never report a
18     false-empty: a lost CAS race retries inside `claim_due_job`, so None means truly
empty.
19
20     The real-timer seam (`_arm_wake_timer`) is monkeypatched to a recorder in every test
that
21     can park a job, so nothing actually sleeps and no daemon timer outlives the test.
22     """
23
24     import sqlite3
25     import threading
26
27     import pytest
28
29     pytest.importorskip("httpx")
30     from fastapi.testclient import TestClient
31
32     import backend.main as m
33     from backend.config.models import AppConfig
34     from backend.infrastructure.database import Database
35     from backend.pipeline.validators.base import ValidationResult
36
37
38     class _InlineExecutor:
39         """submit() runs fn synchronously ? deterministic, single-threaded tests."""
40
41         def submit(self, fn, *args, **kwargs):
42             fn(*args, **kwargs)
43
44
45     @pytest.fixture
46     def env(tmp_path, monkeypatch):
47         db = Database(str(tmp_path / "t.db"))
48         cfg = AppConfig()
49         monkeypatch.setattr(m, "_get_executor", lambda: _InlineExecutor())
50         # Stub the only real-timer seam so a parked job never arms a live daemon timer;
tests
51         # that care about scheduling assert against this recorder instead.
52         armed: list = []
53         monkeypatch.setattr(m, "_arm_wake_timer", lambda delay, *a: armed.append(delay))
54         app = m.create_app(cfg, db)
55         client = TestClient(app)
56         return client, db, tmp_path, monkeypatch, armed
57
58
59     def _video(tmp_path, name="m.mp4"):
60         p = tmp_path / name
61         p.write_bytes(b"not-a-real-video-but-hashable")
62         return p
63
64
65     def _pass():
66         return [ValidationResult(validator_name="x", passed=True, message="ok", details={})]
67
68
69     class _FakeSeg:
70         def __init__(self, db, cfg):
71             self.db = db
72
73         def process_video(self, video_id, video_path, *a, **k):
74             sid = self.db.add_play_segment(video_id, 0.0, 5.0)
75             return [
76                 {
77                     "id": sid,
78                     "start_time": 0.0,

```



```

79         "end_time": 5.0,
80         "duration": 5.0,
81         "metadata": {},
82     }
83 ]
84
85
86 def _wire_happy(mp, client_env_video):
87     mp.setattr(m.registry, "run_all_with_context", lambda *a, **k: (_pass(), {}))
88     mp.setattr(m.segmenter_registry, "get", lambda name: _FakeSeg)
89
90
91     # --- no-regression: the drain runs a queued job exactly like the old submit path
92     -----
93
94     def test_drain_runs_queued_job_to_done(env):
95         client, db, tmp_path, mp, _armed = env
96         vid_file = _video(tmp_path)
97         _wire_happy(mp, vid_file)
98
99         # Submit via the public endpoint (which now submits a drain tick, not _run_job).
100        job_id = client.post(

```

7. user (2026-07-04T08:55:54.389Z)

```

C:\Users\avidu\Projects\kheIsutra-guru\wt-drain-
concurrency\backend\infrastructure\database_user_models.py:9: def _connect(self): # pragma:
no cover - implemented by Database facade
C:\Users\avidu\Projects\kheIsutra-guru\wt-drain-
concurrency\backend\infrastructure\database_user_models.py-10-         raise NotImplementedError
C:\Users\avidu\Projects\kheIsutra-guru\wt-drain-
concurrency\backend\infrastructure\database_user_models.py-11-
C:\Users\avidu\Projects\kheIsutra-guru\wt-drain-
concurrency\backend\infrastructure\database_user_models.py-12- def _execute_write( # pragma:
no cover - implemented by Database facade
C:\Users\avidu\Projects\kheIsutra-guru\wt-drain-
concurrency\backend\infrastructure\database_user_models.py-13-         self, sql: str, params:
Tuple[Any, ...], error_msg: str
C:\Users\avidu\Projects\kheIsutra-guru\wt-drain-
concurrency\backend\infrastructure\database_user_models.py-14-     ) -> bool:
C:\Users\avidu\Projects\kheIsutra-guru\wt-drain-
concurrency\backend\infrastructure\database_user_models.py-15-         raise NotImplementedError
C:\Users\avidu\Projects\kheIsutra-guru\wt-drain-
concurrency\backend\infrastructure\database_user_models.py-16-
C:\Users\avidu\Projects\kheIsutra-guru\wt-drain-
concurrency\backend\infrastructure\database_user_models.py-17-     # --- Per-user model store
(PERSONALIZATION_PLAN.md §2, v5) -----
--
C:\Users\avidu\Projects\kheIsutra-guru\wt-drain-
concurrency\backend\infrastructure\database_media.py:9: def _connect(self): # pragma: no
cover - implemented by Database facade
C:\Users\avidu\Projects\kheIsutra-guru\wt-drain-
concurrency\backend\infrastructure\database_media.py-10-         raise NotImplementedError
C:\Users\avidu\Projects\kheIsutra-guru\wt-drain-
concurrency\backend\infrastructure\database_media.py-11-
C:\Users\avidu\Projects\kheIsutra-guru\wt-drain-
concurrency\backend\infrastructure\database_media.py-12- def _execute_write( # pragma: no
cover - implemented by Database facade
C:\Users\avidu\Projects\kheIsutra-guru\wt-drain-
concurrency\backend\infrastructure\database_media.py-13-         self, sql: str, params:
Tuple[Any, ...], error_msg: str
C:\Users\avidu\Projects\kheIsutra-guru\wt-drain-
concurrency\backend\infrastructure\database_media.py-14-     ) -> bool:
C:\Users\avidu\Projects\kheIsutra-guru\wt-drain-

```

```

concurrency\backend\infrastructure\database_media.py-15-         raise NotImplementedError
C:\Users\avidu\Projects\kheIsutra-guru\wt-drain-
concurrency\backend\infrastructure\database_media.py-16-
C:\Users\avidu\Projects\kheIsutra-guru\wt-drain-
concurrency\backend\infrastructure\database_media.py-17-     def add_video(
--
C:\Users\avidu\Projects\kheIsutra-guru\wt-drain-
concurrency\backend\infrastructure\database.py:32:     def _connect(self):
C:\Users\avidu\Projects\kheIsutra-guru\wt-drain-
concurrency\backend\infrastructure\database.py-33-         """Transaction scope that also CLOSES
the connection.
C:\Users\avidu\Projects\kheIsutra-guru\wt-drain-
concurrency\backend\infrastructure\database.py-34-
C:\Users\avidu\Projects\kheIsutra-guru\wt-drain-
concurrency\backend\infrastructure\database.py-35-         ``with sqlite3.connect(...)`` alone is
only a commit/rollback scope ? it
C:\Users\avidu\Projects\kheIsutra-guru\wt-drain-
concurrency\backend\infrastructure\database.py-36-         never closes, so every call leaked its
connection to GC (lingering file
C:\Users\avidu\Projects\kheIsutra-guru\wt-drain-
concurrency\backend\infrastructure\database.py-37-         handles on the WAL sidecars under
Windows). Internal methods use this.
C:\Users\avidu\Projects\kheIsutra-guru\wt-drain-
concurrency\backend\infrastructure\database.py-38-         """
C:\Users\avidu\Projects\kheIsutra-guru\wt-drain-
concurrency\backend\infrastructure\database.py-39-         conn = self._get_connection()
C:\Users\avidu\Projects\kheIsutra-guru\wt-drain-
concurrency\backend\infrastructure\database.py-40-         try:
--
C:\Users\avidu\Projects\kheIsutra-guru\wt-drain-
concurrency\backend\infrastructure\database_jobs.py:11:     def _connect(self): # pragma: no
cover - implemented by Database facade
C:\Users\avidu\Projects\kheIsutra-guru\wt-drain-
concurrency\backend\infrastructure\database_jobs.py-12-         raise NotImplementedError
C:\Users\avidu\Projects\kheIsutra-guru\wt-drain-
concurrency\backend\infrastructure\database_jobs.py-13-
C:\Users\avidu\Projects\kheIsutra-guru\wt-drain-
concurrency\backend\infrastructure\database_jobs.py-14-     def _execute_write( # pragma: no
cover - implemented by Database facade
C:\Users\avidu\Projects\kheIsutra-guru\wt-drain-
concurrency\backend\infrastructure\database_jobs.py-15-         self, sql: str, params:
Tuple[Any, ...], error_msg: str
C:\Users\avidu\Projects\kheIsutra-guru\wt-drain-
concurrency\backend\infrastructure\database_jobs.py-16-     ) -> bool:
C:\Users\avidu\Projects\kheIsutra-guru\wt-drain-
concurrency\backend\infrastructure\database_jobs.py-17-         raise NotImplementedError
C:\Users\avidu\Projects\kheIsutra-guru\wt-drain-
concurrency\backend\infrastructure\database_jobs.py-18-
C:\Users\avidu\Projects\kheIsutra-guru\wt-drain-
concurrency\backend\infrastructure\database_jobs.py-19-         # --- Async job store
(DEFERRED_HARDENING_PLAN #16) -----
--
C:\Users\avidu\Projects\kheIsutra-guru\wt-drain-
concurrency\backend\infrastructure\database_jobs.py:23:     def create_job(
C:\Users\avidu\Projects\kheIsutra-guru\wt-drain-
concurrency\backend\infrastructure\database_jobs.py-24-         self,
C:\Users\avidu\Projects\kheIsutra-guru\wt-drain-
concurrency\backend\infrastructure\database_jobs.py-25-         job_id: str,
C:\Users\avidu\Projects\kheIsutra-guru\wt-drain-
concurrency\backend\infrastructure\database_jobs.py-26-         video_path: str,
C:\Users\avidu\Projects\kheIsutra-guru\wt-drain-
concurrency\backend\infrastructure\database_jobs.py-27-         segmenter: Optional[str] = None,
C:\Users\avidu\Projects\kheIsutra-guru\wt-drain-
concurrency\backend\infrastructure\database_jobs.py-28-         player_pool: Optional[List[str]]
= None,

```

```

C:\Users\avidu\Projects\khealsutra-guru\wt-drain-
concurrency\backend\infrastructure\database_jobs.py-29-
C:\Users\avidu\Projects\khealsutra-guru\wt-drain-
concurrency\backend\infrastructure\database_jobs.py-30-
= None,
C:\Users\avidu\Projects\khealsutra-guru\wt-drain-
concurrency\backend\infrastructure\database_jobs.py-31-
--
C:\Users\avidu\Projects\khealsutra-guru\wt-drain-
concurrency\backend\infrastructure\database_jobs.py:146:
Optional[Dict[str, Any]]:
C:\Users\avidu\Projects\khealsutra-guru\wt-drain-
concurrency\backend\infrastructure\database_jobs.py-147-
`result`/\`player_pool` JSON deserialized.""
C:\Users\avidu\Projects\khealsutra-guru\wt-drain-
concurrency\backend\infrastructure\database_jobs.py-148-
C:\Users\avidu\Projects\khealsutra-guru\wt-drain-
concurrency\backend\infrastructure\database_jobs.py-149-
C:\Users\avidu\Projects\khealsutra-guru\wt-drain-
concurrency\backend\infrastructure\database_jobs.py-150-
C:\Users\avidu\Projects\khealsutra-guru\wt-drain-
concurrency\backend\infrastructure\database_jobs.py-151-
jobs WHERE id = ?", (job_id,))
C:\Users\avidu\Projects\khealsutra-guru\wt-drain-
concurrency\backend\infrastructure\database_jobs.py-152-
C:\Users\avidu\Projects\khealsutra-guru\wt-drain-
concurrency\backend\infrastructure\database_jobs.py-153-
C:\Users\avidu\Projects\khealsutra-guru\wt-drain-
concurrency\backend\infrastructure\database_jobs.py-154-
--
C:\Users\avidu\Projects\khealsutra-guru\wt-drain-
concurrency\backend\infrastructure\database_jobs.py:260:
-> Optional[Dict[str, Any]]:
C:\Users\avidu\Projects\khealsutra-guru\wt-drain-
concurrency\backend\infrastructure\database_jobs.py-261-
if the job doesn't exist; ``cost_usd`` is ``None`` until
C:\Users\avidu\Projects\khealsutra-guru\wt-drain-
concurrency\backend\infrastructure\database_jobs.py-262-
the deserialized JSON breakdown.""
C:\Users\avidu\Projects\khealsutra-guru\wt-drain-
concurrency\backend\infrastructure\database_jobs.py-263-
C:\Users\avidu\Projects\khealsutra-guru\wt-drain-
concurrency\backend\infrastructure\database_jobs.py-264-
C:\Users\avidu\Projects\khealsutra-guru\wt-drain-
concurrency\backend\infrastructure\database_jobs.py-265-
C:\Users\avidu\Projects\khealsutra-guru\wt-drain-
concurrency\backend\infrastructure\database_jobs.py-266-
C:\Users\avidu\Projects\khealsutra-guru\wt-drain-
concurrency\backend\infrastructure\database_jobs.py-267-
video_id, owner_id, compute_backend, gpu_type, gpu_active_seconds, "
C:\Users\avidu\Projects\khealsutra-guru\wt-drain-
concurrency\backend\infrastructure\database_jobs.py-268-
bytes_out, gemini_cost_usd, pricebook_version, cost_usd, "

```

```

max_frames: Optional[int] = None,

policy: Optional[Dict[str, Any]]

owner_id: Optional[str] = None,

def get_job(self, job_id: str) ->

    """Fetch one job;

    with self._connect() as conn:

        row = (

            conn.cursor()

                .execute("SELECT * FROM

                    .fetchone()

                )

            if not row:

def get_job_cost(self, job_id: str)

    """The job's cost row (``None``

recorded). ``cost_breakdown`` is

    with self._connect() as conn:

        row = (

            conn.cursor()

                .execute(

                    "SELECT id,

                        "wall_seconds,

```

8. user (2026-07-04T08:56:02.819Z)

```

1     import logging
2     import os
3     import shutil
4     import sqlite3
5     from contextlib import contextmanager
6     from typing import Any, Tuple
7
8     from backend.infrastructure.database_jobs import DatabaseJobsMixin
9     from backend.infrastructure.database_media import DatabaseMediaMixin

```

```

10     from backend.infrastructure.database_user_models import DatabaseUserModelsMixin
11
12     logger = logging.getLogger(__name__)
13
14     # The highest schema version this binary knows how to migrate TO (the top of the
migration ladder
15     # in `__init_db`). PACKAGING_PLAN.md P2d uses it for the upgrade-safety contract on a
downloadable,
16     # auto-updating app: refuse to open a DB from a NEWER app (downgrade guard) and back the
DB up before
17     # an upgrade migration runs.
`tests/test_database.py::test_schema_version_matches_ladder` pins it to
18     # the ladder so adding a `_migrate_to_vN` without bumping this fails CI.
19     SCHEMA_VERSION = 9
20
21
22     class SchemaTooNewError(RuntimeError):
23         """The on-disk DB was created by a newer app version than this binary understands
(P2d)."""
24
25
26     class Database(DatabaseMediaMixin, DatabaseJobsMixin, DatabaseUserModelsMixin):
27         def __init__(self, db_path: str):
28             self.db_path = db_path
29             self._init_db()
30
31         @contextmanager
32         def _connect(self):
33             """Transaction scope that also CLOSES the connection.
34
35             ``with sqlite3.connect(...)`` alone is only a commit/rollback scope ? it
36             never closes, so every call leaked its connection to GC (lingering file
37             handles on the WAL sidecars under Windows). Internal methods use this.
38             """
39             conn = self._get_connection()
40             try:
41                 with conn:
42                     yield conn
43             finally:
44                 conn.close()
45
46         def _get_connection(self):
47             # busy_timeout: wait (not fail) on a locked DB ? needed once the async job model
48             # adds concurrent writers (a swallowed 'database is locked' silently drops
segments).
49             conn = sqlite3.connect(self.db_path, timeout=30.0)
50             conn.row_factory = sqlite3.Row
51             # Enforce ON DELETE CASCADE / SET NULL ? SQLite leaves FKs off per-connection.
52             conn.execute("PRAGMA foreign_keys = ON")
53             # WAL allows concurrent readers during a write; busy_timeout bounds lock waits.
54             conn.execute("PRAGMA busy_timeout = 30000")
55             try:
56                 conn.execute("PRAGMA journal_mode = WAL")
57             except sqlite3.OperationalError:
58                 pass # e.g. a read-only/networked FS that can't do WAL ? degrade gracefully
59             return conn
60
61         def _execute_write(self, sql: str, params: Tuple[Any, ...], error_msg: str) -> bool:
62             """Run ONE bool-returning single-statement write inside a connection scope.
63
64             Captures the connect/commit/except-logger.error/return-False boilerplate shared
by
65             the bool-returning single-statement writers. ``error_msg`` is the caller's exact
66             per-method message (already formatted with its own ids) ? logged verbatim on a
67             swallowed write fault so each writer keeps its specific log text. Returns True

```

```

68         success, False (after logging ``error_msg``) on any exception."""
69     try:
70         with self._connect() as conn:
71             conn.cursor().execute(sql, params)
72             conn.commit()
73         return True
74     except Exception as e:
75         logger.error(f"{error_msg}: {e}")
76         return False
77
78     @staticmethod
79     def _add_column_idempotent(cursor: sqlite3.Cursor, ddl: str) -> None:
80         """Apply a bare ``ALTER TABLE ... ADD COLUMN`` migration step re-runnably.

```

9. assistant (2026-07-04T08:56:11.014Z)

The fix diff looks logically sound. Now let me prove the regression test bites at the unfixed PR head.

10. user (2026-07-04T08:56:17.359Z)

Preparing worktree (detached HEAD 8534077)
HEAD is now at 8534077 review: gate concurrency on effective compute path, not the server default (#466)
M tests/test_job_worker_loop.py

11. user (2026-07-04T08:56:32.794Z)

```

Exit code 1
tests/test_job_worker_loop.py | 43 +-----
1 file changed, 42 insertions(+), 1 deletion(-)
    barrier.wait() # release all claimers together so they genuinely race on the head
row
    results[slot] = db.claim_due_job()

    threads = [threading.Thread(target=_claim, args=(i,)) for i in range(n)]
    for t in threads:
        t.start()
    for t in threads:
        t.join()

    claimed_ids = [r["id"] if r else None for r in results]
    # No false-empty: with N runnable jobs, no claimer may come back empty-handed.
> assert all(r is not None for r in results), f"lost-race leaked None: {claimed_ids}"
E   AssertionError: lost-race leaked None: ['j0', 'j1', None, 'j2', None, None, None, None]
E   assert False
E       + where False = all(<generator object
test_concurrent_claimers_claim_all_jobs_distinctly.<locals>.<genexpr> at 0x000001C8CCF501E0>)

tests\test_job_worker_loop.py:258: AssertionError
===== short test summary info =====
FAILED tests/test_job_worker_loop.py::test_concurrent_claimers_claim_all_jobs_distinctly
1 failed in 5.86s

```

12. user (2026-07-04T08:56:50.138Z)

```

..... [100%]
16 passed in 3.71s

```

13. user (2026-07-04T08:57:16.261Z)

```

1 passed in 2.38s
1 passed in 2.36s

```

1 passed in 2.35s
1 passed in 2.49s
1 passed in 2.30s

14. assistant (2026-07-04T08:57:23.231Z)

Test passes 5/5 with the fix. Per the pre-LGTM gate, let me reproduce the full suite, ruff, and mypy claims rather than trust them.

15. user (2026-07-04T08:59:32.224Z)

```
-- Docs: https://docs.pytest.org/en/stable/how-to/capture-warnings.html
1634 passed, 10 skipped, 8 warnings in 121.86s (0:02:01)
All checks passed!
backend\pipeline\validators\base.py:44: note: By default the bodies of untyped functions are not
checked, consider using --check-untyped-defs  [annotation-unchecked]
backend\providers\__init__.py:9: note: By default the bodies of untyped functions are not
checked, consider using --check-untyped-defs  [annotation-unchecked]
Success: no issues found in 144 source files
```

16. user (2026-07-04T08:59:40.651Z)

```
C:\Users\avidu\Projects\khehsutra-guru\wt-drain-concurrency\.github\workflows\nightly.yml:141:
run: python run_all_tests.py --cov=backend --cov=training --cov-report=term --cov-fail-under=70
C:\Users\avidu\Projects\khehsutra-guru\wt-drain-concurrency\.github\workflows\ci.yml:146:
run: python run_all_tests.py --cov=backend --cov=training --cov-report=term --cov-fail-under=70
```

17. user (2026-07-04T09:02:06.976Z)

```
TOTAL                                11529   1682    85%
Required test coverage of 70% reached. Total coverage: 85.41%
1634 passed, 10 skipped, 15 warnings in 137.73s (0:02:17)
=====
RESULT: PASS - all tests green
=====
```

18. user (2026-07-04T09:02:34.876Z)

Structured output provided successfully